

LECTURE 02

Advantages of OOP & the Modern Java Landscape

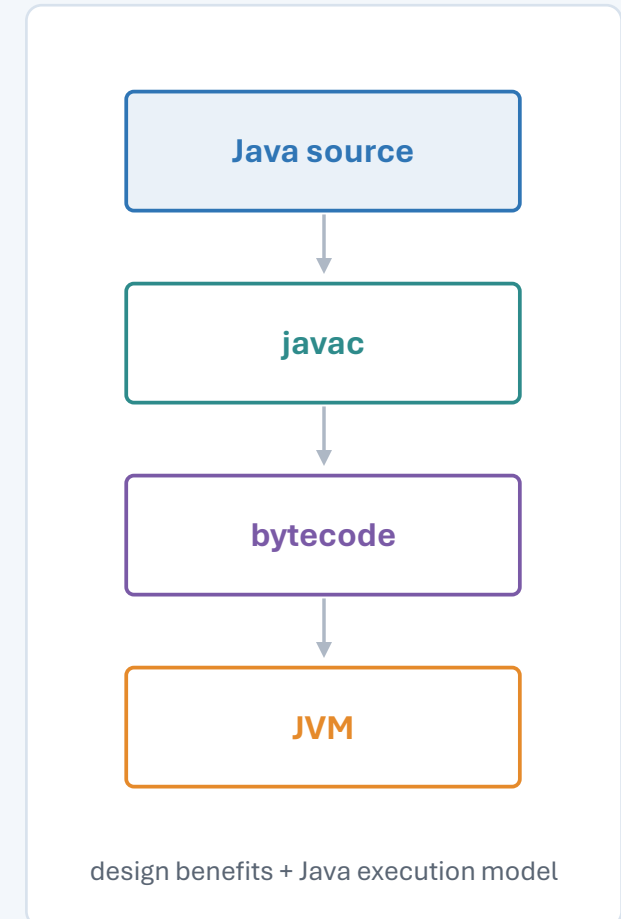
CSC241 — Object-Oriented Programming

Fall 2026 • COMSATS University Islamabad

Instructor: Muhammad Shahid Bhatti

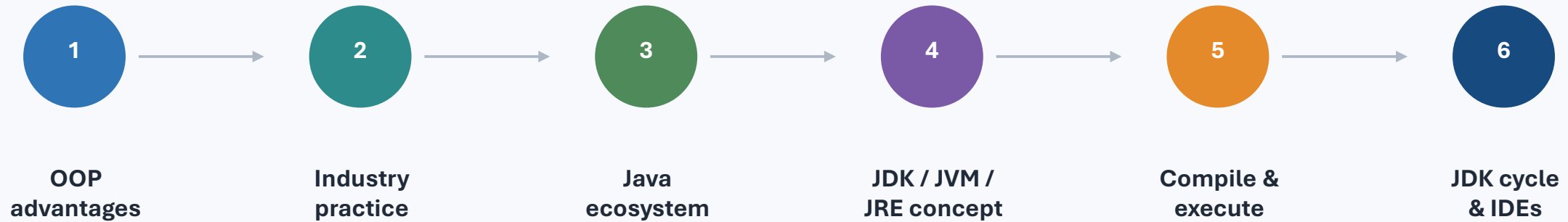
Assistant Professor • Department of Computer Science • Office H-17
Course Repository: github.com/mshahidbhatti/CSC241-OOP

Reference: Lecture plan • Liang, Ch. 1



Lecture roadmap

First: why OOP helps. Then: what actually happens when a Java program runs.



Two questions to keep asking: “Why organize software around objects?” and “What does the Java platform do for us?”

Learning outcomes

By the end of this lecture, you should be able to...

- Explain how OOP can improve reusability, maintainability, scalability, modularity, and extensibility.
- Describe where object-oriented abstractions appear in APIs and frameworks used by modern software.
- Differentiate the Java language, Java SE platform, JVM, JRE concept, and JDK.
- Trace a Java program from .java source through javac, .class bytecode, class loading, and JVM execution.
- Interpret the six-month JDK release cadence and distinguish an LTS release from a non-LTS feature release.

Bridge from Lecture 1

Last time we moved from “functions operating on data” toward “objects owning state and behavior.”

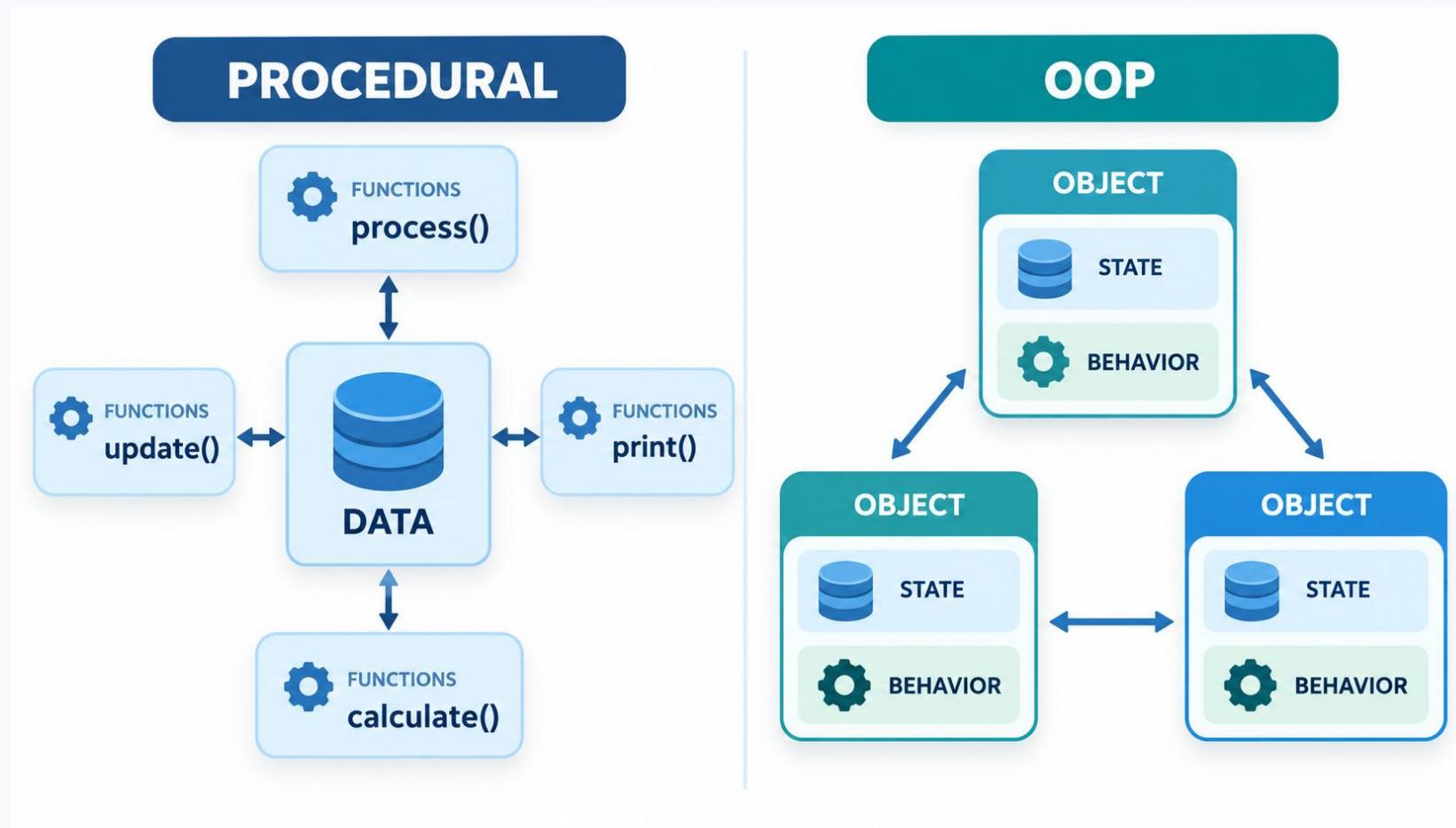


Today: what practical advantages can this reorganization create?

Visual recap — procedural vs. object-oriented organization

VISUAL

This diagram reinforces the organizational contrast carried forward from Lecture 1.



In procedural thinking, functions revolve around shared data. In OOP, state and behavior travel together inside collaborating objects.

The major benefits of OOP

Think of these as design goals—not automatic properties of using the class keyword.

Reuse

avoid re-solving stable problems

Maintain

localize change

Scale

divide a system into parts

Test

verify components in isolation

Extend

add new behavior safely

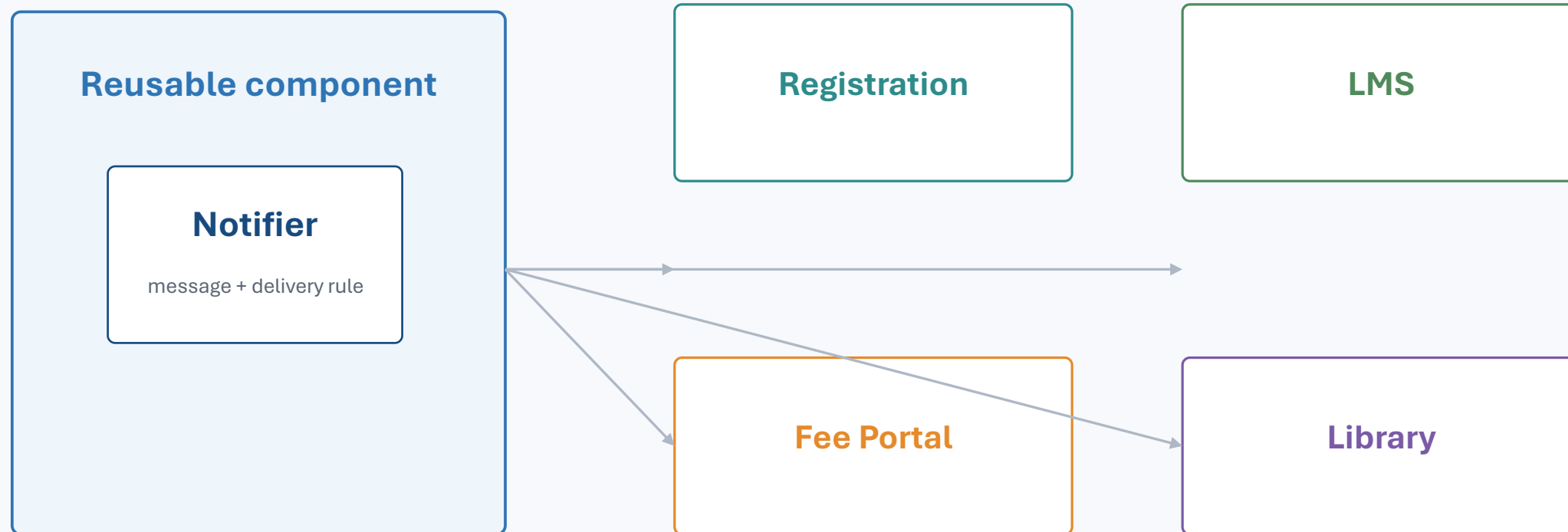
Collaborate

give teams clear boundaries

Good OO design reduces the number of places you must understand and edit when one concept changes.

Advantage 1 — Reusability

Package a useful abstraction once, then use it in many contexts.



Reuse is strongest when the abstraction is cohesive, well-named, and has a stable public contract.

Reuse in Java: use an object-oriented library instead of reinventing it

The Java platform itself is a large collection of reusable types and APIs.

```
import java.time.LocalDate;

public class AddDropDeadlineDemo {
    public static void main(String[] args) {
        LocalDate today = LocalDate.now();
        LocalDate deadline = today.plusDays(7);

        System.out.println("Add/drop closes: " +
        deadline);
    }
}
```

- We reuse `LocalDate` rather than implementing calendar arithmetic ourselves.
- The type exposes meaningful behavior such as `plusDays(...)`.
- Its internal date representation is not our concern.
- The same standard API can support registration, exams, attendance, and deadlines.

Reusability is not only inheritance. Reusing well-designed objects and APIs is often more important.

Advantage 2 — Maintainability

A maintainable design keeps a rule close to the concept that owns it.

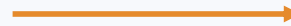
Scattered rules

capacity check in UI

duplicate check in controller

eligibility rule in web form

deadline rule in script



Localized responsibilities

CourseSection.hasSeat()

CourseSection.isEnrolled()

RegistrationPolicy.isEligible()

Semester.isRegistrationOpen()

Maintenance becomes easier when one business concept has one obvious home.

Example: one policy change, one obvious place to edit

Requirement: “A course section must not accept students beyond its capacity.”

```
class CourseSection {  
    private int enrolled;  
    private final int capacity = 40;  
  
    public void enroll() {  
        if (enrolled >= capacity)  
            throw new IllegalStateException("Section is full");  
  
        enrolled++;  
    }  
}
```

Why this helps

- Every registration screen gets the same capacity rule.
- Client code does not need to track remaining seats itself.
- Tests can target enrollment behavior in one place.
- A later policy change has a clear owner.

We will study encapsulation and exceptions formally later. For now, focus on keeping a business rule with the object that owns it.

Advantage 3 — Scalability of the codebase

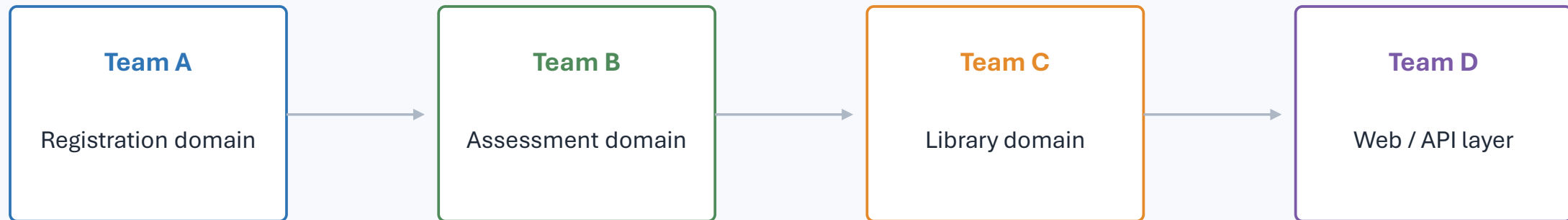
Large systems become manageable when responsibilities are partitioned into coherent components.



Scalability here means organizational scalability: the system can grow without every feature touching every file.

OOP can also scale development across teams

Clear object and module contracts let people work in parallel with fewer accidental dependencies.

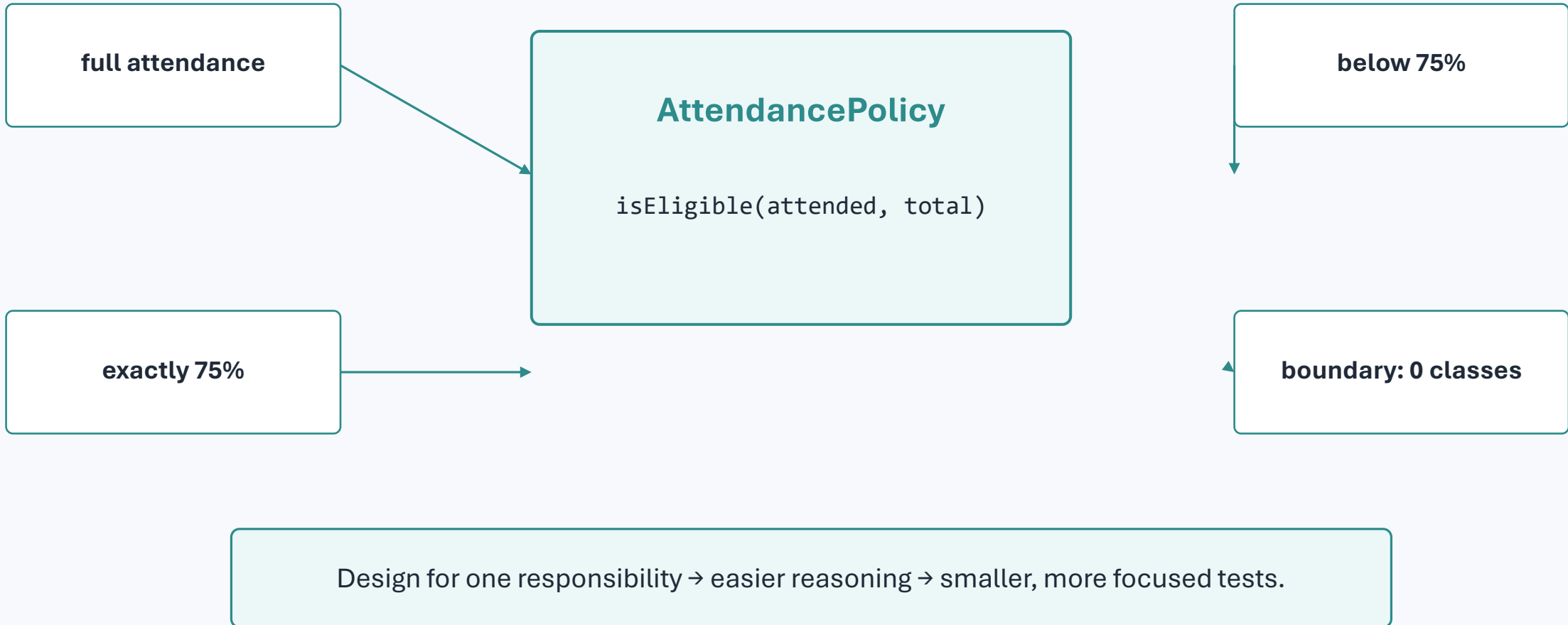


Contracts between components

A good boundary hides implementation details but makes expected behavior explicit.

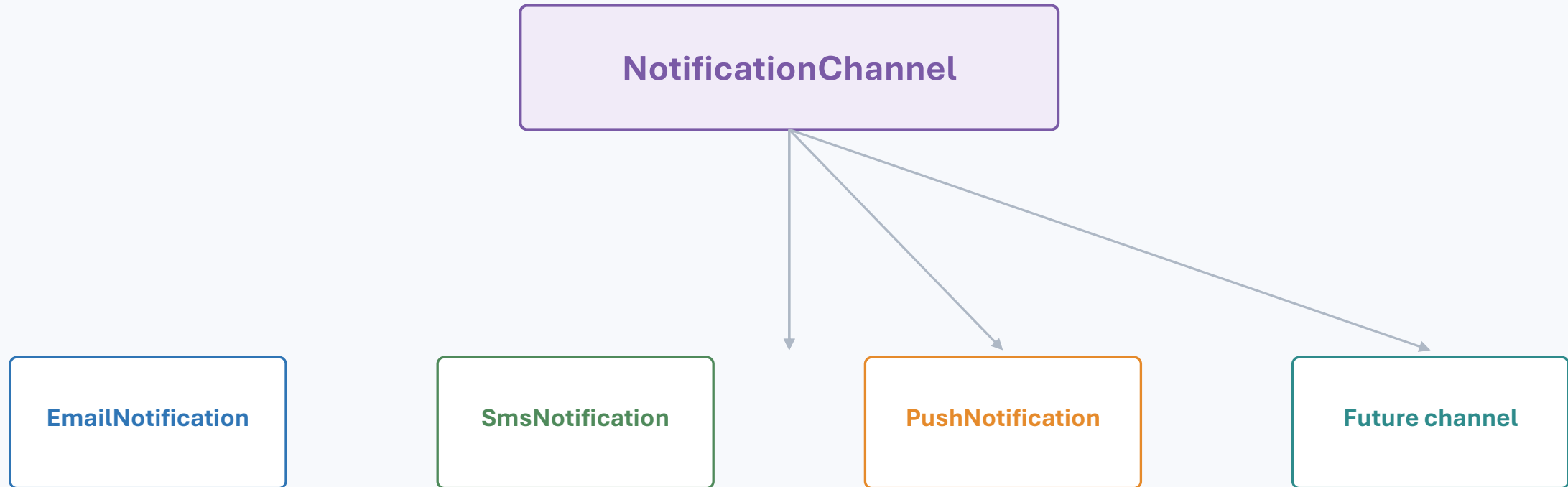
Advantage 4 — Modularity and testability

Small cohesive objects make it easier to test one responsibility at a time.



Advantage 5 — Extensibility

A stable abstraction can allow new variants without rewriting every client.



Later we will implement this idea using inheritance, abstract classes, interfaces, and polymorphism.

OOP does not guarantee good software

Classes can organize complexity—or simply move complexity into more files.

Healthy OO design

- Cohesive responsibilities
- Small, meaningful public APIs
- Hidden internal details
- Low coupling between components
- Composition when appropriate

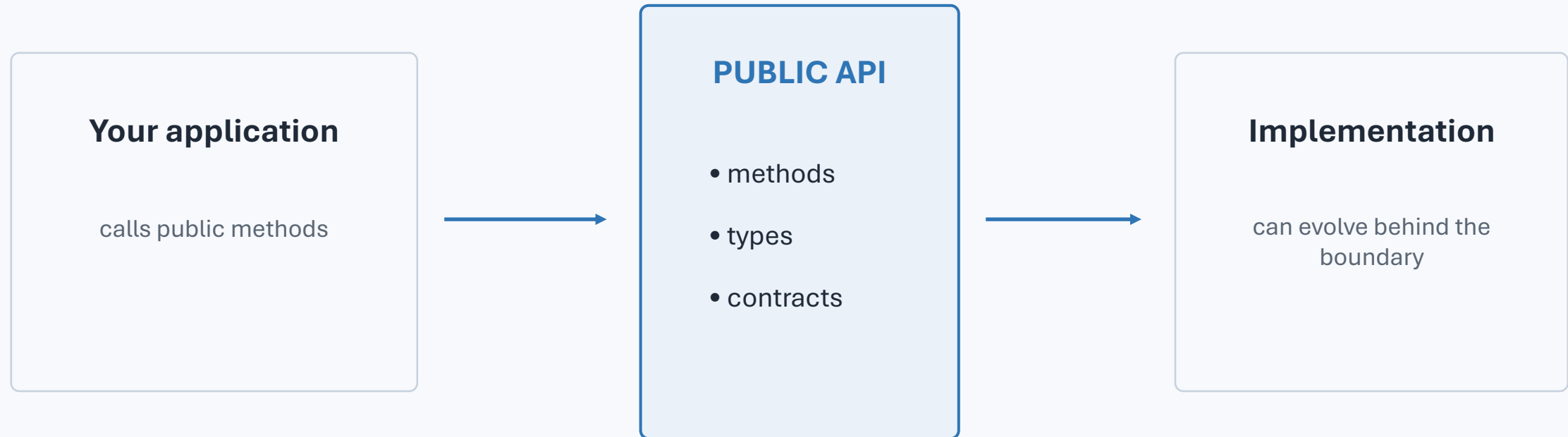
OO warning signs

- “God classes” doing everything
- Getters/setters for every field with no behavior
- Deep inheritance with fragile dependencies
- Objects that merely wrap unrelated procedures
- Too many abstractions for a simple problem

Use OOP when it improves the model and the change structure—not because “everything must be a class.”

Where OOP appears in modern industry practice: APIs

An API exposes objects, operations, and contracts that other code can use without knowing the implementation.



The Java SE library itself is an API: you program against types such as String, List, Path, LocalDate, and many more.

Where OOP appears in modern industry practice: frameworks

A framework supplies reusable structure and invokes your application-specific objects at defined extension points.

Traditional library use

Your code → Library

you call it

Framework-oriented use

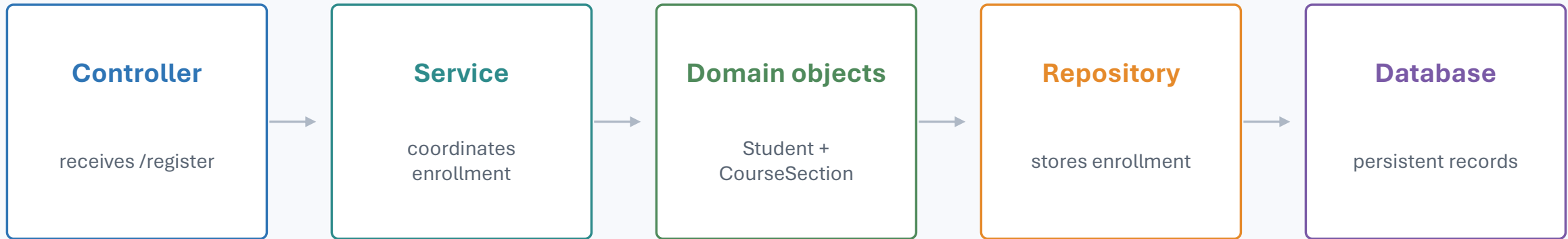
Framework → Your object

framework calls you

Common pattern: framework lifecycle + your domain objects + public contracts.

Example: object collaboration inside a web application

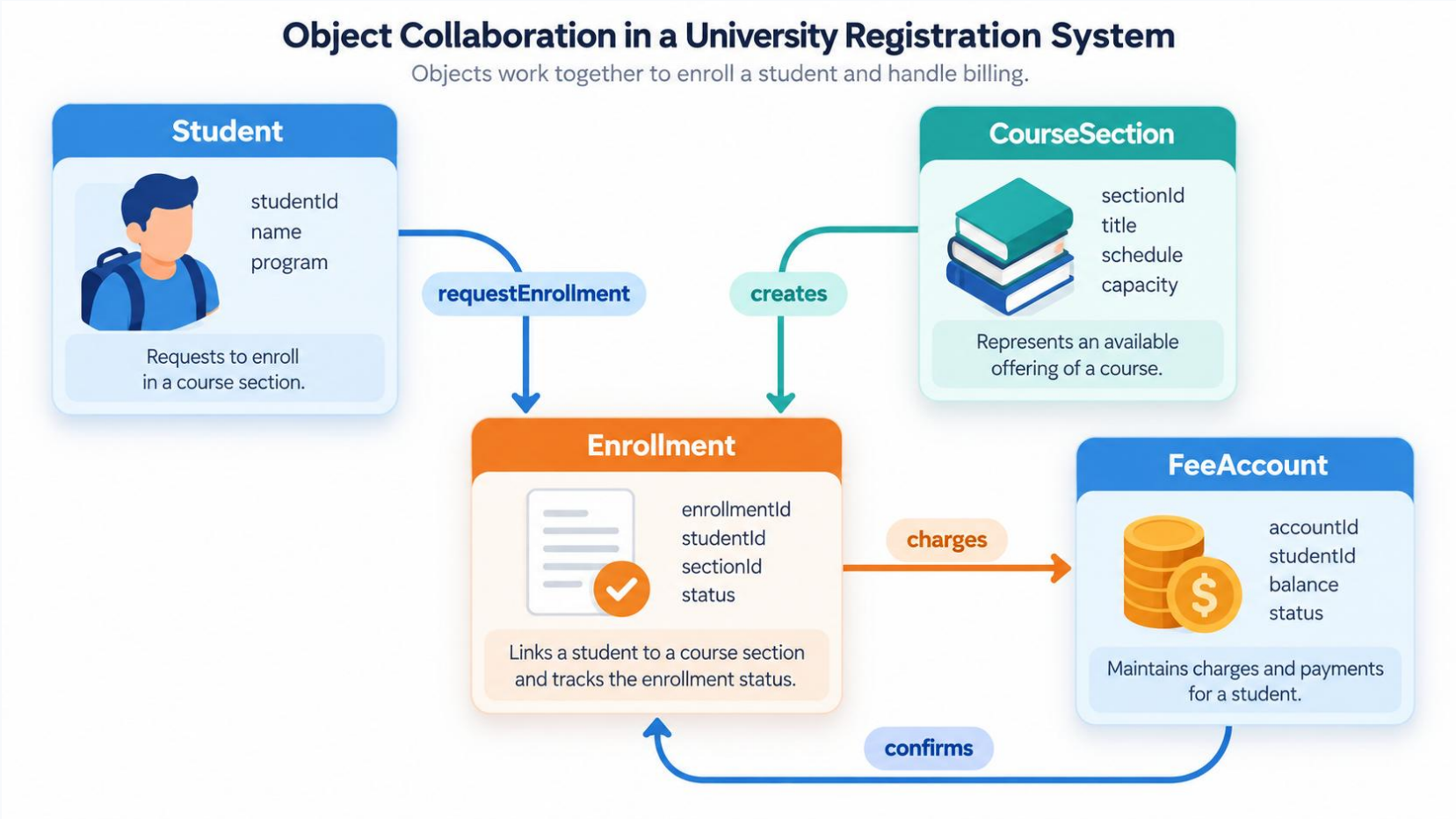
Even when the external API is HTTP/JSON, the internal implementation is often organized as collaborating objects.



OOP gives the application an internal vocabulary: Student, CourseSection, Enrollment, Repository, Service, Controller...

Visualizing object collaboration in a registration system

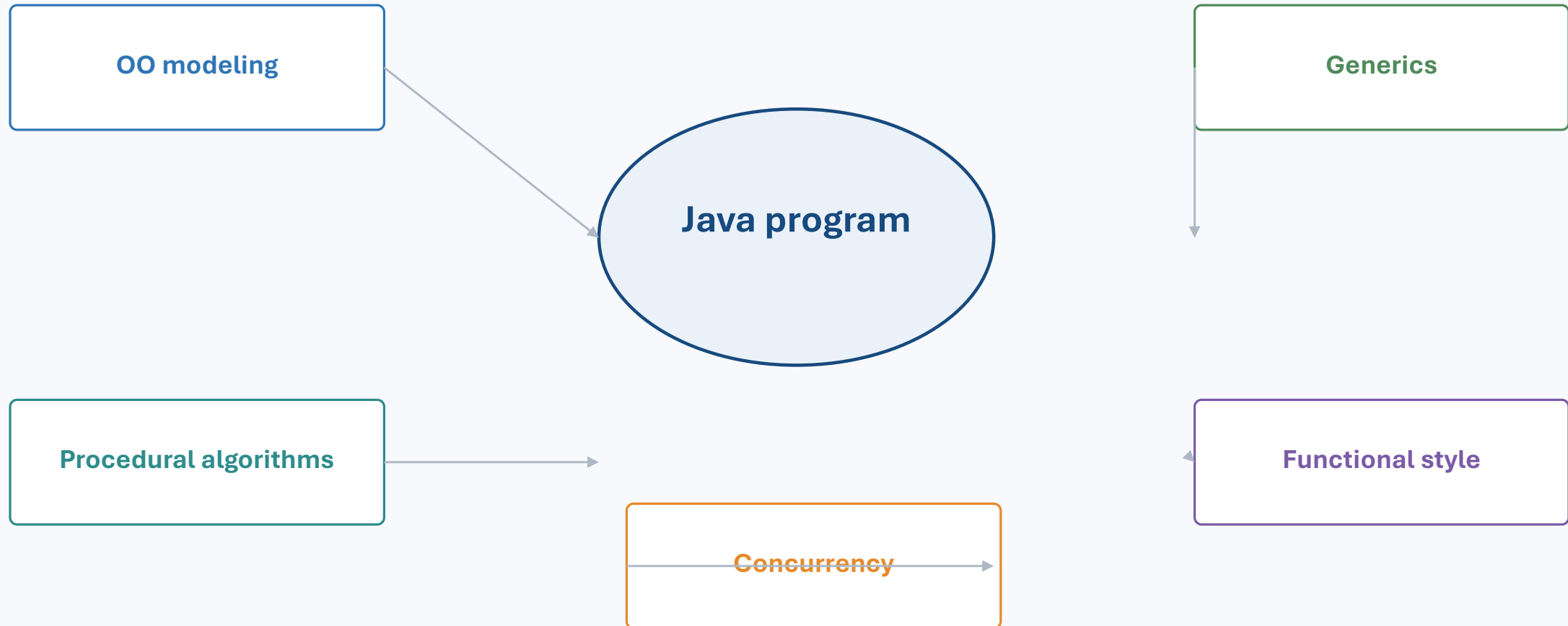
One student request can trigger several collaborating objects — a major reason OOP helps large systems stay manageable.



Notice how each object has a focused role: Student requests, CourseSection offers capacity, Enrollment records the relationship, and FeeAccount handles billing.

Modern Java is object-oriented—but not object-oriented only

Real Java programs mix OO structure with procedural, functional, generic, and concurrent techniques.



Course goal: learn OO design deeply while recognizing that good Java uses multiple complementary styles.

The Java technology landscape

“Java” can refer to a language, a platform, a virtual machine, a development kit, or an ecosystem.

Your Java source code

language syntax + your classes

Java SE APIs

standard libraries and platform contracts

JVM

loads and executes class files

Operating system + hardware

Windows / Linux / macOS / CPU

JDK

development kit

- compiler
- launcher/runtime
- debugger & tools
- standard libraries

Keep the layers separate in your mind: language ≠ JDK ≠ JVM.

Layer 1 — The Java programming language

The language defines syntax, types, expressions, statements, classes, interfaces, and other source-level constructs.

```
public class CourseMessage {  
    public static void main(String[] args) {  
        String course = "CSC241";  
        System.out.println("Welcome to " +  
course);  
    }  
}
```

- What the language specifies: source-level rules.
- Source files normally use the .java extension.
- The language is defined by the Java Language Specification (JLS).
- The source code itself is not what the CPU directly executes.

Think: Java language = the rules you use to write the program.

Layer 2 — Java SE: the standard platform

Java SE combines the core platform specification and a large set of standard APIs available to Java applications.

java.lang

String, Object, Math

java.util

collections, utilities

java.time

date/time API

java.io / java.nio

files & streams

java.net

networking

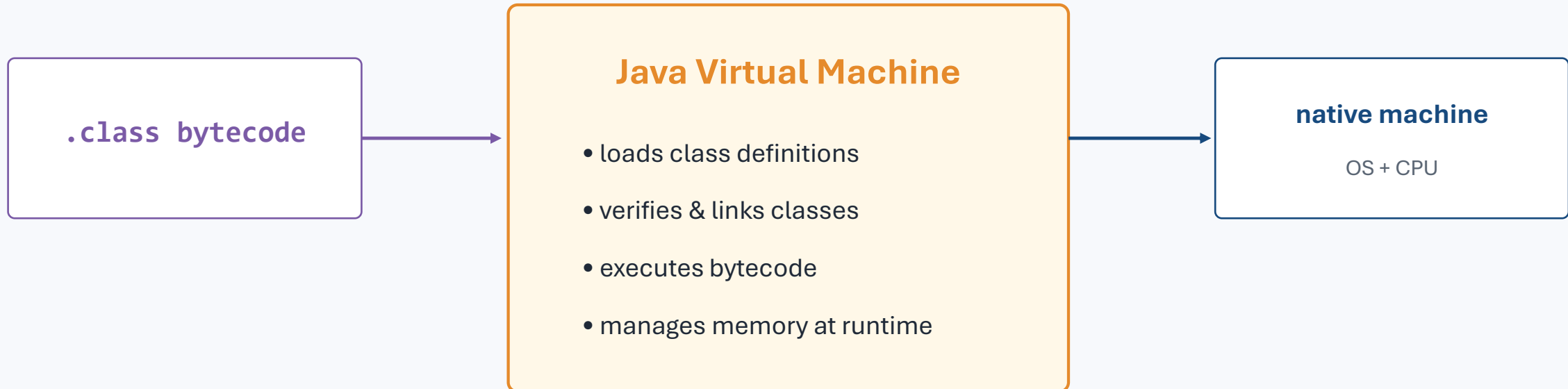
java.sql

database API

Later CSC241 topics—collections, file I/O, serialization, database access—build on these platform APIs.

Layer 3 — JVM: the execution abstraction

The Java Virtual Machine provides a defined execution environment for class files.



The JVM is not “a fake computer” in the simplistic sense; it is a precisely specified runtime execution model.

What happens inside the JVM? — a useful mental model

JVM

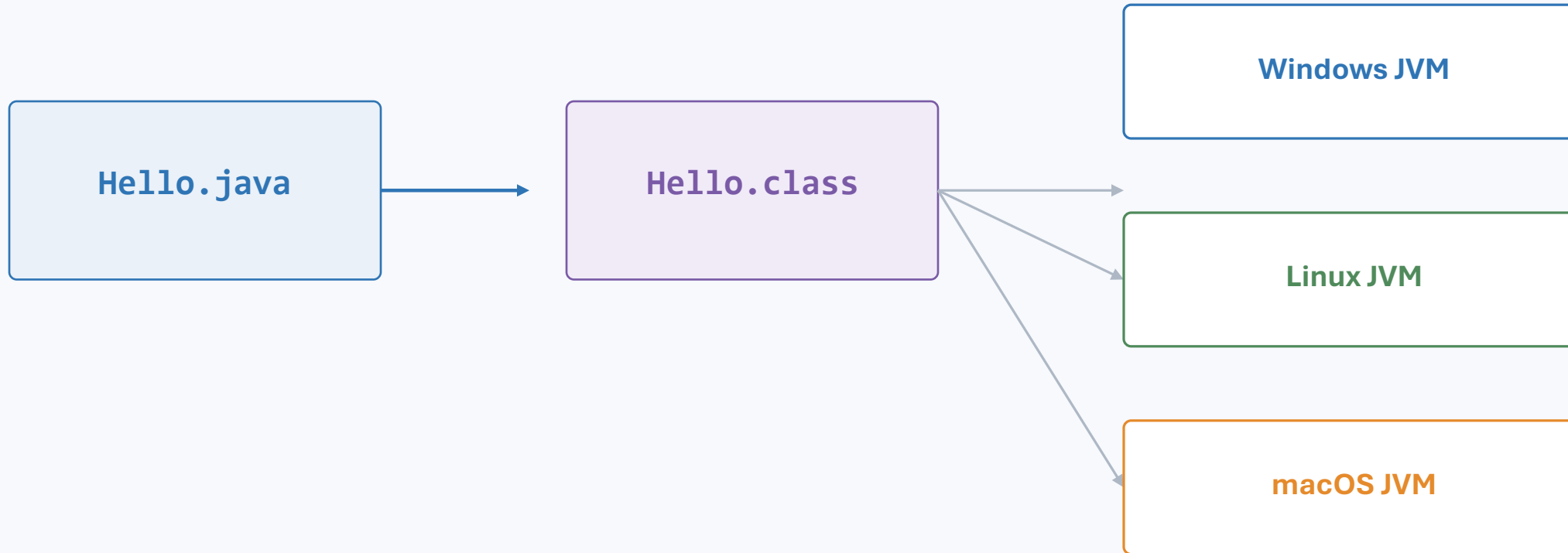
You do not need implementation internals yet; remember the major runtime responsibilities.



Class loading is dynamic: classes are typically loaded as the running program needs them.

Why Java bytecode helps portability

Compile to a standardized class-file format; use a platform-specific JVM implementation to run it.



Nuance: “write once, run anywhere” depends on compatible JVMs, libraries, native dependencies, and environment assumptions.

JRE: an important concept with a modern caveat

Historically, the Java Runtime Environment was the runtime package used to execute Java applications.

Conceptual JRE

JVM + runtime libraries +
supporting runtime components

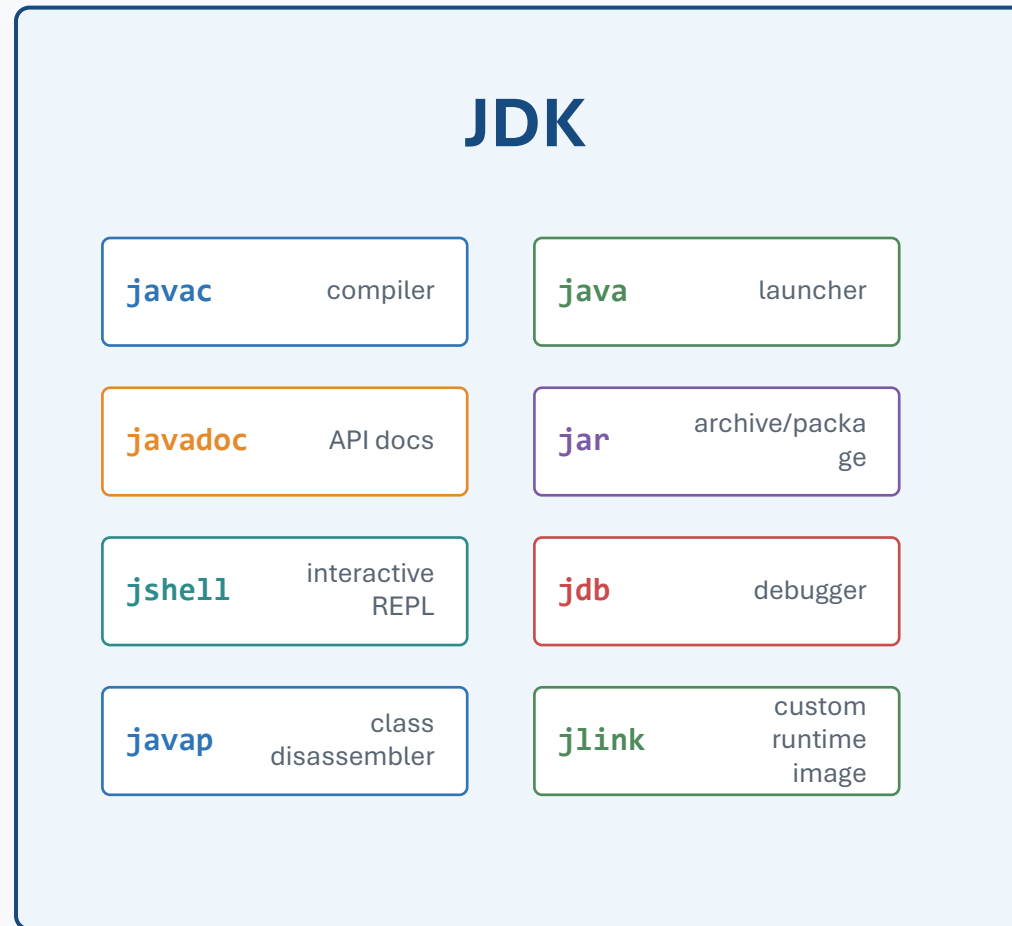
Modern packaging reality

- Since Java 11, Oracle Java releases are offered as JDKs rather than separate JRE/JDK products.
- A JDK contains what developers need to compile and run Java code.
- Application-specific runtimes can be created with tools such as jlink.

For exams: know the JRE concept. For current practice: do not assume you install a separate standalone JRE.

JDK: the developer's toolkit

The Java Development Kit contains the tools needed to develop, inspect, package, and run Java software.



An IDE usually calls these tools for you—but understanding the toolchain makes debugging much easier.

Core JDK commands worth recognizing

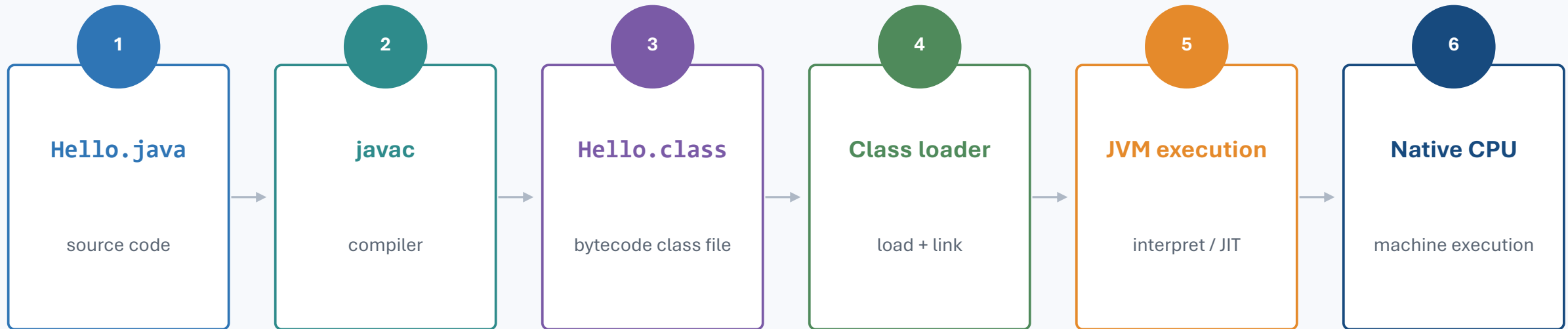
You do not need to memorize every option; understand what each tool is responsible for.

Command	Purpose
javac Hello.java	Compile source to class files
java Hello	Start JVM and run the main class
java -version	Report the selected Java runtime/JDK version
javac -version	Report the compiler version
javap -c Hello	Inspect class structure / bytecode instructions
jar --create ...	Package files into a JAR archive
jshell	Experiment interactively with Java expressions
javadoc ...	Generate API documentation from source comments

The Java source-to-execution pipeline

PIPELINE

This is the central execution story you should be able to draw from memory.



Compile-time catches many source/type errors. Runtime handles loading, dynamic behavior, memory, and execution.

Step 1 — javac compiles .java source into .class files

COMPILE

Compilation checks source-level rules and emits class files for the JVM.

```
// CourseMessage.java
public class CourseMessage {
    public static void main(String[] args) {
        System.out.println("Course: CSC241");
    }
}
```

`javac CourseMessage.java`

javac

CourseMessage.class

Compiler output is a class file—not a Windows .exe.

Source: Oracle javac documentation — javac reads Java source and compiles it into class files that run on the JVM.

© Muhammad Shahid Bhatti

Step 2 — Bytecode is the JVM's instruction format

A .class file is a structured binary format containing JVM instructions and metadata.

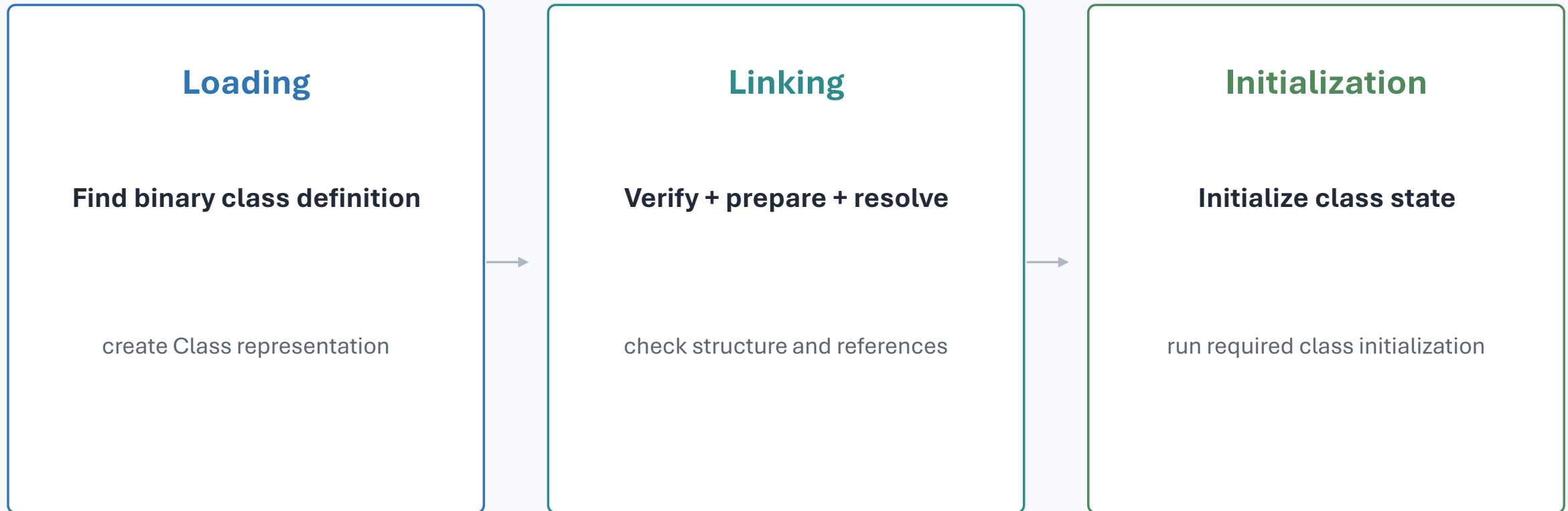
```
Compiled from "CourseMessage.java"
public class CourseMessage {
    public static void main(java.lang.String[]);
    Code:
        0: getstatic      #7      // System.out
        3: ldc            #13     // "Course: CSC241"
        5: invokevirtual #15     // println
        8: return
}
```

- This is a conceptual sample of `javap -c` output.
- Bytecode is not Java source.
- Bytecode is not tied directly to one CPU instruction set.
- The JVM implementation maps execution to the host platform.

Useful debugging tool: `javap -c CourseMessage`

Step 3 — Class loading, linking, and initialization

Before a class can participate in execution, the JVM must obtain and prepare its definition.

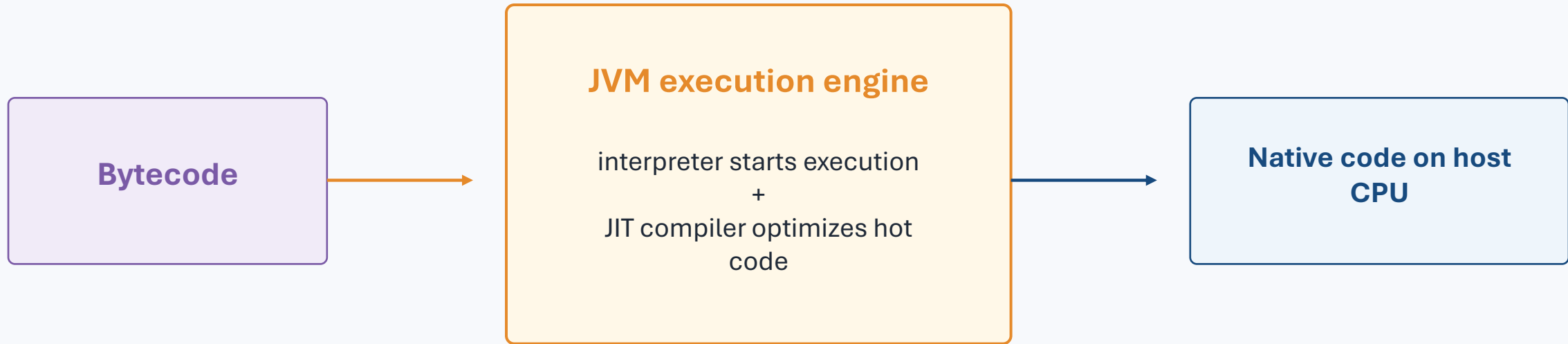


You will see initialization order again when we study constructors, static members, and inheritance.

Step 4 — JVM execution: interpreter + JIT compilation

EXECUTION

A modern JVM can begin executing bytecode and compile frequently executed code into optimized native machine code.



Runtime services around execution

garbage collection

thread scheduling support

exceptions

security/runtime checks

Performance detail varies by JVM implementation; the key concept is that bytecode is executed by a JVM on the host platform.

JDK release cadence — and the Fall 2026 snapshot

Java feature releases follow a six-month cadence; selected releases receive Long-Term Support (LTS).

As of 3 Sep 2026

- JDK 26 is the latest generally available feature release.

Course recommendation

- Prefer an LTS JDK for a semester-long teaching environment.
- JDK 25 LTS is a sensible Fall 2026 baseline unless

TS

As of 3 Sep 2026

- JDK 26 is the latest generally available feature release.
- JDK 25 is the latest LTS release.
- JDK 27 is in Release Candidate phase, with GA scheduled for 15 Sep 2026.

Teaching choice

- Prefer an LTS JDK for a semester-long environment.
- JDK 25 LTS is a sensible Fall 2026 baseline unless your lab standardizes another supported version.
- Newest feature release ≠ best classroom baseline.

Sources: Oracle Java SE Support Roadmap; OpenJDK JDK 27 project schedule; Oracle Java Downloads. Checked 3 Sep 2026.

© Muhammad Shahid Bhatti

Today's Java development environment: JDK + IDE

An IDE improves productivity; the JDK remains the underlying compiler/runtime/toolchain.

IntelliJ IDEA

full Java IDE

Eclipse IDE

mature Java ecosystem

Apache NetBeans

Java-focused IDE

VS Code + Java extensions

lightweight editor workflow

Minimum setup check

```
java -version  
javac -version
```

```
// both should point to the intended  
JDK
```

IDE ≠ JDK

The IDE edits, navigates, builds, debugs, and integrates tools. The JDK provides the Java compiler/runtime toolchain underneath.

For the course: use one consistent JDK and verify the IDE project SDK before writing code.

Lecture recap & exit ticket

You should now be able to explain both the design motivation for OOP and the Java execution pipeline.

Remember

- OOP can localize change and responsibilities.
- APIs/frameworks use object contracts as boundaries.
- Java language, Java SE, JVM, JRE concept, and JDK are different things.
- javac produces .class files for the JVM.
- JDK feature releases arrive every six months; LTS is a support designation.

Exit ticket

- Draw the Java source-to-execution pipeline from memory.
- What is the difference between the JVM and the JDK?
- Why is a separate JRE less central in modern Java distributions?
- Give one concrete maintainability benefit of OOP.
- Why might a course choose JDK 25 LTS rather than JDK 26?

Next lecture: Classes and Objects — anatomy of a class, instantiation, fields, methods, this, identity, state, and references.